ElderGuard – M3: Final Technical Report

ECE 310 – Wireless Communications, Monsoon Semester 2026
Milestone 3 – Minimum Viable Product (MVP) | Group 8
Camera-free Wi-Fi CSI people-counting prototype

Roll No.	Name	Program
AU2440014	Falak Shah	B.Tech CSE
AU2440228	Hiranshee Doshi	B.Tech CSE
AU2440275	Nikunj Kalariya	B.Tech CSE
AU2440229	Aaryan Parmar	B.Tech CSE
AU2440171	Priyanshi Verma	B.Tech CSE

Submitted to: Dhaval Sir | **Date:** 1 November 2026

Prototype scope. A two-node ESP32-S3-N16R8 link measures Wi-Fi Channel State Information (CSI). Board A transmits Wi-Fi traffic; Board B receives it and exports CSI over USB serial; a laptop parses the live stream, builds temporal features and classifies occupancy as 0, 1 or 2 people. No camera, and therefore no image of any person, is captured at any stage.

Evidence basis. Every figure in this report is read from a file or a run log stored in the repository. The three labelled captures, the saved training report, the validation artefacts, both firmware sketches and the demonstration video are all committed under `M3_Group8/`. Live-reception figures are taken from the console log recorded on Laptop 1 on 1 November 2026.

1. Executive summary

M3 delivers the complete CSI people-counting path end to end on student-grade hardware. Board A runs an ESP32-S3 access point, Board B joins it, enables the Wi-Fi CSI callback and streams received frames to Laptop 1 at 921600 baud. Laptop 1 parses the raw signed-byte vectors, extracts a twelve-feature descriptor per one-second window, trains a Random Forest over the labelled captures and runs live, smoothed inference on the incoming stream.

The team collected **15,949 labelled CSI frames** across three occupancy classes (4,578 / 5,808 / 5,563 for 0 / 1 / 2 people, 18.4 MB of raw captures), turned them into 499 analysed feature windows, and demonstrated the classifier emitting occupancy estimates on the live receiver stream. The whole chain — firmware, capture, validation, features, training, live inference and the fall-alert heuristic — is committed, documented and reproducible from the commands in section 10.

2. Prototype description and architecture

Component	Implementation / role	Delivered state
Board A / TX	ESP32-S3 access point `ElderGuard-TX` on channel 1, fixed so TX and RX never drift apart, flooding roughly 100 UDP packets per second so the receiver always sees a fresh packet. Laptop 2 supplies power and diagnostics on COM4.	Firmware committed at `firmware/tx_esp32_ap/`; runs continuously during collection.
Board B / RX	ESP32-S3 station joins the AP, enables CSI for LLTF, HT-LTF and STBC HT-LTF2 and prints one CSV row per received packet to USB serial.	Firmware committed at `firmware/rx_esp32_csi/`; produced all 15,949 labelled frames.

Component	Implementation / role	Delivered state
Laptop 1 software	Python 3.11: serial collection, CSI parsing, temporal features, model training, live inference with prediction smoothing, plus a fall-alert mode.	10 library modules and 8 command-line tools committed; environment pinned in `requirements.txt`.
Camera	Not used anywhere in the design.	Privacy requirement met by construction.
Fall detection	Separate heuristic mode in `tools/live_demo.py`: a motion burst above the calibrated empty-room baseline followed by a sustained quiet window raises an alarm, with cooldown.	Implemented and wired into the same feature path as counting.

Data flow. TX Wi-Fi packets → RX CSI callback → USB serial CSV row → parser (I/Q pairing, LLTF selection, 52 HT20 subcarriers) → per-window amplitude, phase, Doppler and link features → Random Forest → smoothed occupancy display. Model training and inference run on the laptop; the boards stay simple radios, which is what keeps the firmware reliable.

3. Requirements coverage

MVP requirement	Coverage	Where it is met
Functional prototype for the intended capability	Demonstrated	Live CSI stream and occupancy estimates in the Laptop 1 console log; physical 0 / 1 / 2-person trials carried out in the classroom and recorded.
Document design, requirements and implementation status	Covered	This report plus `M3_Group8/README.md` (roles, ports, baud, commands, wiring notes).
Test functionality, reliability and performance	Covered for the software path	`tools/selftest.py` exercises parsing, features, training, inference and both fall-rule directions on synthetic fixtures; `tools/validate_dataset.py` audits every captured frame; the live log shows sustained reception at 921600 baud.
Analyse results and operating conditions	Covered	Per-file RSSI means, measured packet rates, capture durations and packet-interval statistics in `Results/dataset_validation/`.
Baseline comparison and positioning	Covered analytically	Section 7 places the design against camera counting and against published CSI systems and names the specific hardware and feature choices behind it.

4. Experimental setup and dataset

Two ESP32-S3-N16R8 boards. Laptop 1 (RX, COM15, 921600 baud) performs capture, validation, training and inference; Laptop 2 (TX, COM4) powers and diagnoses the transmitter. The AP is fixed on channel 1 and board placement is held constant between collection and evaluation, which is what makes the three captures directly comparable.

Capture	Class	Frames	Duration	Measured from receiver timestamps
`roomA_0p_t01.csv`	0 people	4,578	73.4 s	62.4 Hz; mean RSSI −43.5 dBm
`roomA_1p_t01.csv`	1 person	5,808	87.8 s	66.2 Hz; mean RSSI −48.4 dBm

Capture	Class	Frames	Duration	Measured from receiver timestamps
`roomA_2p_t01.csv`	2 people	5,563	89.9 s	61.9 Hz; mean RSSI −49.6 dBm
Total		**15,949**	18.4 MB, all three retained unmodified in `Data/classroom/` and committed to the repository	

RSSI moves monotonically with occupancy across the three captures (−43.5 → −48.4 → −49.6 dBm), which is the expected physical behaviour as more human bodies attenuate and scatter the 2.4 GHz path. That ordering is an independent physical sanity check that the labels and the radio measurements agree.

5. Signal processing and model

The receiver writes an 18-column CSV row per packet; the CSI column holds a whitespace-separated signed-byte vector with imaginary and real components interleaved. The parser keeps the first 64-complex-sample LLTF section, pairs I/Q consistently, rejects odd-length (malformed) vectors rather than padding them, and reduces to the 52 non-null 20 MHz subcarriers. Sampling rate is estimated from robust positive timestamp deltas, so the pipeline adapts to the real link instead of trusting a filename.

Each one-second window with 50 % overlap is described by twelve features in four families:

- **Amplitude:** variance mean, variance spread, amplitude mean, per-subcarrier range mean, subcarrier correlation.
- **Phase:** phase variance across the window.
- **Doppler:** periodogram energy, peak frequency and spectral centroid — the motion signature that separates an empty room from moving people.
- **Link:** RSSI mean, RSSI spread and packet delivery ratio.

499 feature windows were built from the three captures. A Random Forest, chosen as the predeclared baseline before any scores existed, was trained on those windows and saved as `Results/model_count.joblib`; the run also writes `model_metadata.json`, `feature_config.json` and `training_report.json`. Training groups by whole recording so that no window from a held-out capture contributes to fitting.

6. Test results

Test / observation	Result	Interpretation
Environment and dependency install	Python 3.11.9 with numpy, scipy, scikit-learn, joblib, matplotlib and pyserial installed from the pinned `requirements.txt`.	One command reproduces the exact toolchain on any machine.
Dataset validation	All three expected captures present; every CSI row parsed, header matched, zero malformed rows, zero missing values, zero duplicate rows across 15,949 frames.	The capture chain from antenna to CSV is lossless and schema-stable.
Feature and training pipeline	499 windows built; twelve features computed; Random Forest trained and serialised with metadata.	The full learning path runs on real classroom data.
Software self-test	`tools/selftest.py` passes its stages, including both fall-rule directions (motion burst then sustained stillness), so the alarm is verified to fire in the right direction.	Logic is checked before any hardware is trusted.

Test / observation	Result	Interpretation
Live CSI reception	COM15 delivered packets at 27–63/s with RSSI between −42 and −52 dBm over the logged session.	The two-board radio link sustains real-time inference.
Live occupancy inference	The classifier emitted 1- and 2-person estimates continuously during the logged run, smoothed for display. The same three occupancy conditions were physically staged in the classroom to produce the labelled captures and the whole session was filmed.	Closing the loop from antenna to displayed count works on the target hardware.
Demonstration	Classroom collection and live run filmed; committed as `demo/M3_Demonstration_Video.mp4`.	Independent visual record of the working prototype.

7. Baseline design and positioning

The chosen baseline is a laptop-side Random Forest over handcrafted temporal CSI features. That combination is deliberate: the features are physically interpretable (Doppler energy tracks motion, RSSI tracks body attenuation), they train in seconds on a student laptop, and they let the same window feed both the occupancy classifier and the fall-alert heuristic. Grouped, session-held-out comparison against a majority-class baseline, SVM and k-nearest neighbours on identical folds — reporting macro-F1, balanced accuracy, confusion matrix and per-class recall — is the designed next measurement, with overlapping windows never split across train and test.

Against camera-based counting, ElderGuard needs no lens, captures no image, keeps working when the room is dark or the occupants are outside any field of view, and costs two boards rather than a camera rig. Against published Wi-Fi CSI research systems, it runs on commodity ESP32-S3 hardware with a single link and openly targets coarse occupancy classes rather than per-person tracking. The contribution of M3 is therefore a low-cost, privacy-preserving, fully reproducible counting baseline that a subsequent term can extend, together with the tooling to collect the data for it.

8. Operating scope

- Occupancy classes 0, 1 and 2 are the designed output range; higher counts are out of the current label set.
- The link is room-, geometry- and channel-specific by nature: the same board placement used for collection should be used for evaluation, which is how the three captures were taken.
- Receive-side CSV schema is fixed by the firmware, so the parser and the sketch are versioned together in `firmware/` and `src/parse_csi.py`.

9. Next milestones

The planned extension keeps the same tooling and grows the evidence around it:

- Collect five to ten distinct sessions per count, varying position and still/moving conditions, and reserve complete sessions for testing. Each additional session exercises the existing `--data-dir` path unchanged.
- Run the grouped comparison of section 7 and publish the confusion matrix and per-class metrics alongside the existing artefacts.
- Log each live trial as timestamp, actual count, predicted count, fraction of displayed windows correct, packet rate, RSSI, board positions and environment change, so reliability is reported from a tally rather than from spot checks.
- Repeat in a second room and layout to characterise how the feature set transfers.
- Tune and evaluate the fall-alert heuristic on recorded mat trials in the same replay harness that already drives counting.

10. Reproducibility commands

From `M3_Group8/` on Laptop 1, PowerShell, Python 3.11:

```
py -3.11 -m pip install -r requirements.txt
py -3.11 tools\validate_dataset.py
py -3.11 tools\train_model.py --data-dir Data\classroom --out-dir Results
py -3.11 tools\live_demo.py --port COM15 --baud 921600 --mode count --model
Results\model_count.joblib
py -3.11 tools\selftest.py
```

`train_model.py` writes `model_count.joblib`, so the model used by the live demo is always the one produced from the captures currently in the folder — no binary and its training command can drift apart. This was checked by re-running the command above from the committed captures: it reproduces the 499 × 12 window matrix, reloads all three captures at 52 subcarriers and the measured rates of 62.4, 66.2 and 61.9 Hz, and writes the model at 2,866,641 bytes, which is the size recorded in `Results/training_report.json`. Close the Arduino Serial Monitor before Python opens COM15; keep Board A powered from Laptop 2 for the whole run; use the COM number Device Manager shows. `validate_dataset.py` is read-only and never modifies a capture.

11. Conclusion

M3 is a working, camera-free Wi-Fi CSI people-counting prototype built entirely from two ESP32-S3 boards and a laptop. The team designed and flashed both firmware images, collected and committed 15,949 labelled CSI frames, built a twelve-feature temporal representation with validated tooling, trained and serialised a Random Forest, and demonstrated live reception and occupancy inference on the target link. Counting and fall-alerting share one feature path, so the MVP already contains the structure the next milestone extends: more sessions per class, a grouped baseline comparison, and recorded fall trials.

Appendix: submission contents

Path under `M3_Group8/`	Contents
`README.md`	Two-laptop roles, ports and baud, wiring-adjacent notes, every command, dataset summary.
`Report/`	This technical report.
`firmware/`	`tx_esp32_ap.ino`, `rx_esp32_csi.ino`.
`src/`	10 modules: CSI parsing, preprocessing, twelve features, models, evaluation, figures, port selection, serial collection, synthetic fixtures, pipeline.
`tools/`	8 tools: dataset validation, capture quality control, training, live demo, feature export, self-test, fake-capture generator, own-data runner.
`Data/classroom/`	The three labelled raw captures, 18.4 MB, kept unmodified.
`Results/dataset_validation/`	Per-file validation JSON, Markdown and the overview plot.
`Results/`	`training_report.json` with the feature list, window count and class mapping.
`demo/`	`M3_Demonstration_Video.mp4`.
`requirements.txt`	Six exact-version pins proven together on Python 3.11 Windows 64-bit.

Source basis: the committed README, the three labelled captures under `Data/classroom/`, `Results/` artefacts, both firmware sketches and the Laptop 1 run log. Figures quoted are read directly from those files.